Component 2 – SQLi XAI Novelty Page Explained

Oct 22, 2026 · @Kavin Hiruna

Overview

The SQLi XAI novelty page explains why the trained SQL injection model flagged one network request, and how far that explanation can be trusted. It does not stop at "the model predicted SQL injection". It shows which characters in the request caused the decision, which attack technique it is, what would change the decision, and whether different explanation methods agree.

Code lives in two files: `sqli_xai_novelty_page.py` (the screen, in `dev/frontend/pages/com2_attack/`) and `sqli_xai_engine.py` (the XAI logic). Colours and fonts are in `dev/backend/component2_xai/sqli_xai_styles.py`.

What the page does, step by step

Each visit runs the same six steps, from loading files to exporting the result.

Step	What happens	Where on screen
1. Load	Reads the EdgeIIoT dataset, the trained model (`sql_injection_rf.pkl`) and the feature list	Sidebar
2. Build engine	Creates the XAI engine and a background sample of about 300 real rows (needed by SHAP, LIME and the random baseline)	Spinner
3. Select incident	Filters to requests with SQL-like text; the analyst searches or picks an Incident ID	Top
4. Predict	Turns the request into 54 features and gets the real model's prediction and probability	4 KPI cards
5. Explain	Runs the five XAI parts (N1 to N5 below)	Rest of page
6. Export	Downloads the full explanation as JSON	Bottom

What each main function does

The page file handles the screen; the engine file does all model and XAI work.

Page file (`sqli_xai_novelty_page.py`)

Function	Job
`first_existing()`	Finds the first dataset or model file that exists on disk
`load_csv()`	Reads the dataset once and caches it, so reloads are fast
`load_artifact()`	Loads the model or feature list from an upload or the default path
`candidate_mask()`	Quick text filter: keeps rows with `'`, `UNION`, `SLEEP`, `--` and similar. A filter only, not a classifier

Engine file (`sqli_xai_engine.py`)

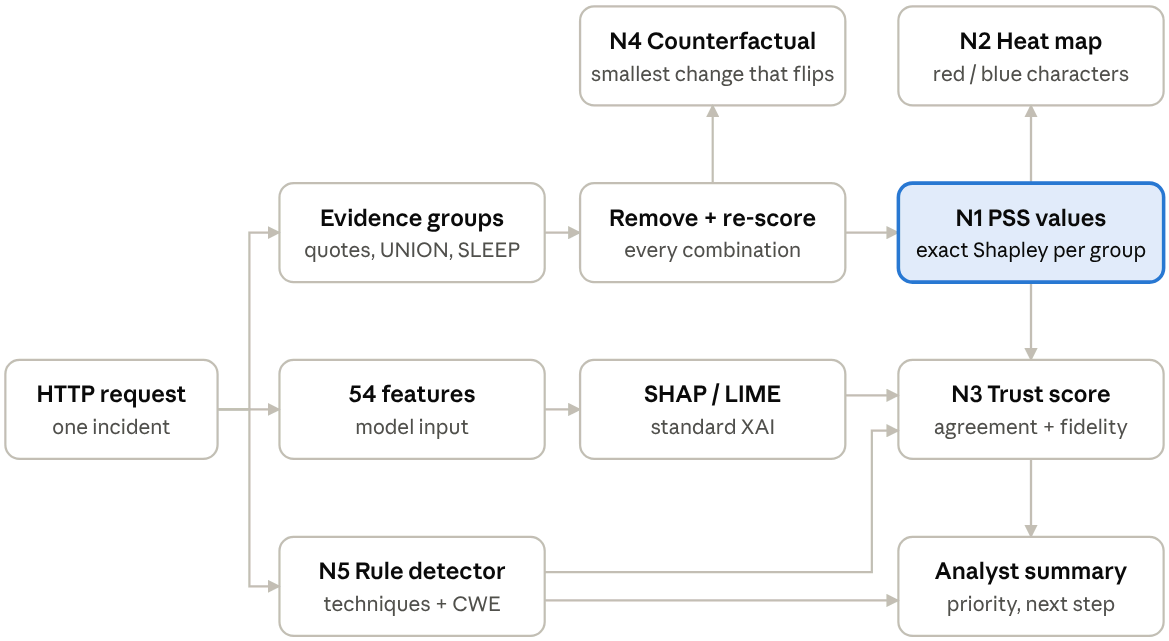
Function	Job
`ColumnResolver.view()`	Collects the request text (URI, body, referer), URL-decodes it (several rounds if needed) and reads the TCP numbers
`default_feature_builder()`	Calculates the 54 features; can also calculate them with chosen evidence removed, which makes the novelty possible
`find_spans()`	Finds the exact character positions of every piece of SQL evidence
`payload_span_shapley()`	N1: exact Shapley values per evidence group
`char_attribution()`, `render_attribution_html()`	N2: the coloured heat map

Function	Job
`shap_values()`, `lime_weights()`	Standard SHAP and LIME, used for comparison
`_trust()`, `_deletion_test()`	N3: Explanation Trust Score
`_counterfactual()`	N4: minimal change that flips the decision
`detect_techniques()`, `build_narrative()`, `_priority()`	N5: technique, standards mapping, analyst summary and priority
`explain()`	Runs everything above for one incident and returns one result object

Where the novelty is

Component 2 ties every explanation to the real request text and then checks whether the explanation can be trusted. Typical XAI-IDS work runs SHAP or LIME and shows a bar chart of feature names such as `single_quote_count` or `tcp.srcport`. That leaves three gaps:

- The analyst cannot see where in the request the problem is.
- Nobody checks whether the explanation is correct.
- It does not say what attack it is or what to do next.



How the five novelty parts fit together

The request feeds three paths: payload evidence (N1, N2, N4), standard SHAP / LIME, and rules (N5). The trust score (N3) compares them before the analyst summary is written.

N1. Payload-Span Shapley (PSS) — the main novelty

SHAP explains abstract feature columns; PSS explains real parts of the request text.

1. The request is split into evidence groups: quotes, comments, AND/OR, UNION, SLEEP, `=` and `()`, and so on.
2. For every combination of groups, those characters are removed from the text, the features are recalculated and the model is asked again.
3. Shapley's formula gives each group its fair share of the decision.

The values are exact, not approximate, because there are at most 12 groups. They add up exactly to the difference between P(attack) with and without the SQL evidence.

N2. Character-level heat map

PSS values are painted onto the exact characters of the request: red pushes towards attack, blue pushes away. The analyst reads the request itself, not a feature list.

N3. Explanation Trust Score

This answers "should I believe this explanation?" by combining four checks:

- Rules ↔ PSS: do the detected techniques match what drove the model?
- PSS ↔ SHAP and PSS ↔ LIME: do the methods agree on the top evidence?
- Deletion fidelity: does removing the top evidence drop the probability more than removing the same number of random characters?

The result is High, Medium or Low. When trust is Low, the page warns the analyst to verify manually before acting.

N4. Minimal sanitisation counterfactual

The engine finds the smallest set of evidence groups whose removal changes the decision to benign, for example "removing only parentheses / equals signs lowers P to 46%". If no combination flips the decision, the page reports that the model relies on network or size features rather than the SQL payload, which is a finding about the model itself.

N5. Technique profiling and analyst narrative

- Detects UNION-based, boolean-blind, time-blind, error-based, stacked queries, enumeration, comment truncation, quote break-out and obfuscation (including double URL-encoding).
- Maps each to CWE-89, CAPEC-66/7 and MITRE ATT&CK T1190, with a recommended next step.
- Writes a plain-English summary and assigns a priority: Critical, High, Medium or Low.

What the example incident already shows

In the test incident (`keywords=' or 'a' 'a`), only about 21% of the attack probability comes from the SQL payload; the model leans on network fields.

- PSS: P(attack) is 44% with all SQL evidence removed and 56% with it, so the payload adds 12 points, (56 − 44) / 56 ≈ 21% of the decision.
- SHAP: the largest contributions are `tcp.dstport` and `tcp.srcport`, not SQL features.
- Counterfactual: removing only parentheses / equals signs lowers P(attack) to 46%.

Standard SHAP alone does not make this weakness obvious; PSS, payload share and the counterfactual together do. This is useful thesis evidence that Component 2 exposes real weaknesses in the IDS model.

Limitations and next steps

The results are only valid if the feature builder matches how the model was trained.

- Feature builder: if it differs from the training preprocessing, predictions differ. Get the model author's feature code and pass it as `feature_builder=`.
- Overlapping groups: some evidence belongs to two groups (for example, `SELECT` is both a keyword and part of UNION SELECT).
- Design choices: the trust thresholds (0.7 / 0.4) and technique severities need justification in the thesis.

- [ ] Run `evaluate_sqli_xai.py` on 300 or more incidents and report the averages
- [ ] Compare with related XAI-IDS papers
- [ ] Optional: a small analyst user study